

RLHF, DPO e Modelo Determinístico

Conceitos aplicados ao projeto **Proton Flow** (TOTVS)

Como este documento foi feito: li o repositório inteiro (backend FastAPI, frontend React, scripts, testes, dataset e documentação) e todas as afirmações sobre o projeto vêm do código atual. Onde o código e o README divergem, está indicado. Onde há recomendação ou estimativa minha, está marcado como **proposta**. Interpretei "FDO" como **DPO** (Direct Preference Optimization).

1. Resumo executivo

- **O projeto já é majoritariamente determinístico.** O LLM (Ollama) faz só uma passada de extração; todo o resto (gatilhos, urgência, filtro de tarefas, prazos, deduplicação, recomendações TOTVS, ata executiva) é regra em Python.
- **A parte probabilística não está travada:** a chamada ao Ollama usa `temperature: 0.1` sem seed nem `top_k`, e o **modelo muda conforme o hardware** (perfil auto: `qwen2.5:3b` sem GPU, `llama3` com GPU). Hoje a mesma reunião pode dar resultados diferentes.
- **Já existe a metade "HF" do RLHF e a estrutura de dados do DPO:** o painel Confirmar/Editar/Rejeitar e a trilha de auditoria coletam feedback humano, e o registro já separa o que a IA sugeriu (`suggested_value`) do que o humano decidiu (`confirmed_value`), que é o formato de um par `chosen/rejected`. O que falta é o treino (recompensa/DPO) e fazer o feedback voltar para a IA (seção 8).
- **RLHF/DPO ainda não têm dados para rodar:** o dataset tem 114 sugestões de campos geradas pela IA e **todas** estão com `review_status = pending`. Ninguém confirmou/editou/rejeitou ainda, então não existe nenhum par de preferência.
- **Recomendação (proposta):** (1) tornar a inferência determinística agora (mudança de poucas linhas); (2) começar a registrar o feedback humano que a interface já permite; (3) só depois, com dados suficientes, avaliar DPO. RLHF completo (PPO) não se justifica para este projeto.

2. O projeto em números

Item	Valor (verificado no repositório)
Produto	Proton Flow: transforma reuniões (ao vivo ou históricas) em atas, dores, tarefas, indicadores trimestrais e recomendações de produtos TOTVS
Backend	Python + FastAPI + Pydantic v2 + Uvicorn; faster-whisper (STT); 46 decorators de rota HTTP (alguns são aliases) e 2 WebSockets
Frontend	React 19 + Vite 8, Recharts, jsPDF, docx; maiores telas: LiveMeetingPage (~3.450 linhas), App (~2.460), QuarterlyAnalyticsPage (~1.330)
IA	Ollama local, 1 chamada por bloco de transcrição, resposta em JSON
Persistência	Arquivos JSON com escrita atômica e lock (<code>dataset_limpo.json</code> , <code>perfis_clientes.json</code> , <code>audit_events.json</code>) — sem banco de dados
Testes	149 funções <code>def test_</code> em backend/tests (unit + integration) e 2 testes Node no frontend
Dataset atual	44 reuniões; mediana ~21 mil caracteres, máximo ~122 mil; 19 analisadas pela IA (11 com <code>qwen2.5:3b</code> , 8 com <code>llama3</code>), 23 aguardando análise
Produção da IA	Nas 19 analisadas: 22 tarefas e 27 dores extraídas; 100% com <code>analysis_engine = ollama</code> (nenhuma caiu no fallback)

3. Como a IA funciona hoje (pipeline real)

Arquivo principal: `backend/analysis_service.py` (~1.900 linhas). O fluxo de `AnalysisService.analyze()` é:

Etapa	O que faz	Tipo
1. Gatilhos	Regex (<code>PADROES_GATILHO</code>) detecta frases como "aguardando aprovação", "prazo vencido", "não vai dar tempo" e já classifica categoria/severidade/sistema TOTVS	Determinístico
2. Divisão em blocos	<code>dividir_transcricao</code> : cabe em 2 (cpu) ou 3 (gpu) blocos. Reuniões longas são amostradas (início + fim [+ miolo]), não lidas por inteiro	Determinístico
3. Chamada ao LLM	POST no Ollama com <code>SYSTEM_PROMPT_V2</code> + gatilhos + transcrição entre tags <code><untrusted_meeting_transcript></code> ; <code>format: json</code>	Probabilístico
4. Fusão	<code>consolidar_resultados</code> : junta blocos, deduplica, mantém a maior severidade (sem nova chamada ao modelo)	Determinístico
5. Checagem de evidência	<code>validar_evidencias_contra_transcricao</code> : se o trecho citado não existe na transcrição, confiança cai para 0,30	Determinístico
6. Regras de negócio	<code>aplicar_regras_deterministas_seguranca</code> : eleva urgência por palavras-chave, filtra tarefas-ruído, normaliza prazos relativos, escolhe responsável por pontuação, injeta gatilhos nas dores	Determinístico
7. Ata executiva	<code>build_executive_summary(use_llm=False)</code> monta a ata só a partir dos dados já validados	Determinístico
8. Recomendação TOTVS	<code>totvs_catalog.compute_recommendations</code> : regras + mapa de segmentos + tratamento de negação ("não temos problema de...")	Determinístico
Fallback	Se o Ollama cair ou devolver JSON inválido, <code>_build_deterministic_fallback</code> gera a análise só com regras e marca <code>analysis_engine = deterministic_fallback</code> + revisão humana obrigatória	Determinístico

Segurança contra prompt injection

A transcrição é tratada como dado não confiável (tags delimitadoras + instrução explícita no prompt), validada por `tests/unit/test_prompt_security.py`. Isso é um ponto forte: o LLM nunca grava nada direto; tudo passa por Pydantic e pelas regras.

4. Auditoria de determinismo do projeto

Determinístico significa: mesma entrada, mesma saída. Abaixo, onde o projeto já garante isso e onde ainda não.

Ponto	Situação atual (código)	Impacto
Parâmetros do LLM	<code>temperature: 0.1</code> ; sem seed, <code>top_k</code> ou <code>top_p</code> (<code>analysis_service.py</code> , payload do Ollama)	Saídas podem variar entre execuções
Escolha do modelo	Perfil auto detecta GPU; o modelo muda (<code>qwen2.5:3b</code> vs <code>llama3</code>). O dataset já contém análises dos dois	Mesma reunião, máquinas diferentes = resultados diferentes
Versão do modelo	Tag sem digest (<code>llama3 = "latest"</code>)	Atualizar o modelo no Ollama muda os resultados sem aviso
Formato de saída	<code>format: "json"</code> (JSON livre, não o schema completo)	Chaves podem faltar/variarem; parse falha vira fallback

Ponto	Situação atual (código)	Impacto
Amostragem da transcrição	Reuniões longas viram só 2-3 blocos	Determinístico, mas ignora parte da reunião
Pós-processamento	Regras Python, Pydantic, deduplicação	Já determinístico
Cache	Não encontrei cache por hash de entrada	Cada "reanalisar" chama o LLM de novo
Divergência de docs	README diz perfil cpu = num_ctx 4096 / bloco 4.000 / timeout 300s; o código usa 8192 / 7.500 / 180s (e max_chunks=2)	Documentação desatualizada

5. Como tornar o modelo determinístico no Proton Flow

5.1 Parâmetros de decodificação

Parâmetro	Valor	Efeito
temperature	0	Escolhe sempre o token mais provável (greedy)
top_k	1	Considera só o token mais provável
top_p	1.0	Sem corte aleatório
seed	fixo (ex.: 42)	Fixa o gerador aleatório
num_predict / num_ctx	fixos	Tamanho de resposta e contexto não mudam entre execuções

5.2 Mudança proposta (analysis_service.py, payload)

```

payload = {
    "model": self.model_name,
    "prompt": texto_enriquecido,
    "system": SYSTEM_PROMPT_V2,
    "stream": False,
    "format": MEETING_JSON_SCHEMA,      # schema completo em vez de "json"
    "options": {
        "num_ctx": OLLAMA_NUM_CTX,
        "num_predict": OLLAMA_NUM_PREDICT,
        "temperature": 0,                # hoje: 0.1
        "top_k": 1,
        "top_p": 1.0,
        "seed": 42,
    },
}
# MEETING_JSON_SCHEMA pode ser gerado de MeetingAnalysisResult.model_json_schema()

```

5.3 Reprodutibilidade fora dos parâmetros

- **Fixar o modelo:** definir OLLAMA_MODEL explicitamente no .env (a variável já tem prioridade sobre o perfil) e registrar o *digest* do modelo em analise_metadados. Sem isso, o perfil auto continua trocando de modelo.
- **Cache por hash:** chave = sha256(modelo + digest + versão do prompt + texto do bloco). O projeto já tem prompt_version e catalog_version nos metadados, então a chave é fácil de montar. Garante repetição 100% mesmo se a GPU variar.
- **Teste de repetibilidade:** novo teste que roda a mesma transcrição N vezes (com Ollama mockado e, opcionalmente, real) e compara o JSON final.
- **Atualizar o README** com os valores reais dos perfis.

Limite honesto: mesmo com temperature 0 e seed fixo, o resultado pode mudar em GPUs/versões diferentes do Ollama (aritmética de ponto flutuante). Determinismo total só com o cache (5.3).

6. RLHF — Reinforcement Learning from Human Feedback

Humanos **ranqueiam** respostas do modelo; o modelo é otimizado para gerar as mais bem ranqueadas.

- **1. Preferências:** para o mesmo prompt, o modelo gera 2+ respostas e uma pessoa escolhe a melhor.
- **2. Modelo de recompensa:** uma rede aprende a dar nota imitando essas escolhas.
- **3. RL (normalmente PPO):** o LLM é ajustado para maximizar a nota, com uma **penalidade KL** para não se afastar demais do modelo original.
- **Custo:** 3-4 modelos ao mesmo tempo (policy, referência, recompensa, valor), instável, exige GPU grande.

```
prompt -> LLM (policy) -> resposta -> Reward Model -> nota
          ^                                     |
          +---- atualização PPO (+ penalidade KL) <-----+
```

7. DPO — Direct Preference Optimization

Mesmo objetivo do RLHF, **sem modelo de recompensa e sem RL**: usa direto os pares (resposta boa, resposta ruim) numa função de perda simples.

```
L = -log sigmoid( beta * [ log(pi(yw|x)/pref(yw|x)) - log(pi(yl|x)/pref(yl|x)) ] )
yw = resposta preferida    yl = resposta rejeitada
pi = modelo treinado       pref = modelo de referência congelado
beta = quanto o modelo pode se afastar da referência (típico 0.1)
```

Critério	RLHF (PPO)	DPO
Modelo de recompensa	Sim, separado	Não (implícito)
Aprendizado por reforço	Sim	Não
Modelos em memória	3-4	2 (treinado + referência); com LoRA, menos
Estabilidade	Sensível	Estável
Dado necessário	Prompts + rankings	Triplas prompt / chosen / rejected
Adequação ao Proton Flow	Baixa (custo e complexidade)	Média, se houver dados (seção 9)

8. O que o Proton Flow já faz de RLHF e de DPO

Resposta curta (vale para RLHF e para DPO): o projeto implementa a **primeira metade do ciclo** (a IA propõe, o humano decide e tudo fica registrado), mas **não fecha o ciclo**: nenhuma decisão humana volta para melhorar o modelo. Não existe modelo de recompensa nem treino por reforço. Por isso o correto é chamar de **human-in-the-loop (HITL)**, a base sobre a qual um RLHF poderia ser construído.

8.1 Mapa: etapas do RLHF x o que existe no código

Etapa do RLHF	No Proton Flow	Status
Gerar resposta com o modelo	AnalysisService.analyze gera sugestões (field_suggestions) e recomendações TOTVS via Ollama	Existe
Humano avalia a resposta	Painel Confirmar / Editar / Rejeitar (MeetingFieldSuggestions.jsx); Confirmar / Rejeitar nas recomendações (TotvsProductRecommendations.jsx); status de revisão da ata executiva (pending_review, reviewed, approved)	Existe (parcial)
Registrar a avaliação	update_suggestion e update_recommendation_status gravam status, valor confirmado, autor, data e histórico; evento em audit_events.json; recomendações espelhadas no SQLite (tabela recommendations)	Existe
Aplicar a decisão humana	Valor confirmado sobrescreve o campo oficial (RESPONSAVEL_REUNIAO, NIVEL_URGENCIA, tema); a decisão humana prevalece sobre a IA; reanálise preserva recomendações já confirmadas/rejeitadas	Existe
Sinal de qualidade automático	validar_evidencias_contra_transcricao (evidência inventada vira confiança 0,30), filtro de ruído em tarefas com rejected_tasks_audit, confiança numérica por item	Existe (equivale a uma recompensa por regras, mas não é usada para treinar)
Modelo de recompensa	Nenhum	Não existe
Otimização do modelo (PPO/DPO)	Nenhuma; o código do Ollama só faz inferência	Não existe
Feedback volta para a IA	Busquei confirmed_value e review_status em analysis_service.py, totvs_catalog.py e analytics_service.py: só aparecem na criação do estado "pending". Nenhum lê o que o humano decidiu	Não existe

8.2 O que isso significa na prática

- **Coleta:** pronta para o fluxo de campos e recomendações. O ciclo de revisão (STATUS_REVISAO = revisao_pendente) existe para as 42 reuniões que têm o campo.
- **Uso do feedback:** hoje serve só para **corrigir o registro** daquela reunião e para auditoria, não para ensinar o modelo.
- **Perfil do cliente:** o main.py injeta no prompt as "dores recorrentes" do cliente, mas esse histórico é calculado a partir das análises da própria IA, não do que humanos corrigiram. É contexto (in-context), não aprendizado por feedback.
- **Volume:** as 114 sugestões do dataset estão todas pendentes, então ainda não há feedback acumulado (detalhes na seção 9.2).

Frase para usar com o grupo: "O Proton Flow já coleta feedback humano estruturado e auditável (a etapa 'HF' do RLHF) e guarda, lado a lado, a sugestão da IA e a decisão humana (o formato de par do DPO), mas ainda não usa isso para treinar ou ajustar o modelo. A coleta existe; o aprendizado não."

8.3 E de DPO? O que o projeto já tem

DPO treina com **pares** (resposta preferida x resposta rejeitada para o mesmo prompt). O projeto **não treina DPO** (busquei dpo, chosen, fine-tune, reward, rlhf em todo o código Python/JS e não há ocorrência), mas já produz, por efeito do fluxo de revisão, os **ingredientes** de um par: o que a IA disse e o que o humano decidiu para o mesmo campo.

Ingrediente do DPO	Onde existe no projeto	Status
Prompt (x)	Transcrição da reunião (ANON_TRANSCRICAO) + contexto do cliente, montados em <code>analyze()</code> . O prompt exato não é salvo , mas é reconstruível a partir da transcrição + <code>SYSTEM_PROMPT_V2</code>	Reconstruível
Resposta rejeitada (y_l)	<code>suggested_value</code> quando o humano edita/rejeita; recomendações com <code>review_status = rejected</code> ; tarefas em <code>rejected_tasks_audit</code> (texto original + motivo da rejeição, 'origem: llm')	Existe, sem dados ainda
Resposta preferida (y_w)	<code>confirmed_value</code> após <i>Editar</i> (valor digitado pelo humano); recomendações confirmadas	Existe, sem dados ainda
Par completo (x, y_w , y_l)	Só nasce na ação Editar de um campo. <i>Confirmar</i> gera só positivo; <i>Rejeitar</i> gera só negativo	Possível, nenhum exportado
Rastreabilidade do par	Autor, data, valor antigo/novo em <code>audit_events.json</code> ; histórico por recomendação	Existe
Exportador para formato DPO	Nenhum (não há <code>prefs.json/JSONL</code> de preferências)	Não existe
Treino (DPOTrainer/LoRA) e modelo de referência	Nenhum; o modelo no Ollama nunca é re-treinado	Não existe
Avaliação antes/depois	Nenhuma métrica de taxa de edição humana por versão de prompt/modelo (embora <code>prompt_version</code> e <code>model</code> já sejam gravados em <code>analise_metadados</code>)	Não existe

- **Ponto forte:** o desenho de dados já separa `suggested_value` (IA) de `confirmed_value` (humano) no mesmo registro. É exatamente a estrutura `chosen/rejected`, sem precisar mudar o schema para os campos atuais.
- **Limite 1:** o `update_suggestion` grava *Editar* como `confirmed`, igual a *Confirmar*. Dá para separar comparando os valores, mas o status "edited" previsto no schema nunca é usado.
- **Limite 2:** tarefas e dores não passam pelo painel, então o feedback mais útil para o modelo (extração de tarefas) hoje só aparece como descarte automático por regra, não como preferência humana.
- **Limite 3:** as tarefas descartadas por regra são negativos *automáticos*: dizem o que a regra não aceita, não o que um humano prefere. Servem como complemento, não substituem pares humanos.

8.4 Como fechar o ciclo, do mais simples ao mais completo (proposta)

Nível	Como usar o feedback	Treina o modelo?
A. Exemplos no prompt (few-shot)	Inserir no prompt 2-3 correções humanas recentes do mesmo cliente/segmento	Não
B. Regras	Transformar correções recorrentes em regras (ex.: novo termo em TERMOS_URGENCIA_ALTA)	Não
C. Fine-tuning supervisionado (SFT)	Treinar com os valores confirmados/editados como resposta correta	Sim
D. DPO	Treinar com pares (editado pelo humano = chosen, sugestão da IA = rejected)	Sim
E. RLHF completo	Modelo de recompensa + PPO	Sim (custo alto; não recomendado aqui)

Os níveis A e B são baratos e dão retorno rápido; C e D só fazem sentido com volume de feedback (seção 9).

9. RLHF/DPO aplicados ao Proton Flow

9.1 O feedback humano que a interface já captura

O projeto já tem o fluxo certo: o painel de validação permite **Confirmar**, **Editar** ou **Rejeitar** cada sugestão da IA (MeetingFieldSuggestions.jsx -> POST /api/meetings/{id}/suggestions/{campo}/confirm -> MeetingRepository.update_suggestion). Cada sugestão guarda suggested_value (o que a IA disse) e confirmed_value (o que o humano decidiu), e a ação vai para audit_events.json. Recomendações TOTVS têm fluxo parecido (confirmed/rejected), e tarefas descartadas ficam em rejected_tasks_audit.

Ação do usuário	Dado gerado	Serve para DPO?
Confirmar	suggested_value == confirmed_value	Só como exemplo positivo (SFT); não forma par
Editar	suggested_value (IA) != confirmed_value (humano)	Sim: chosen = humano, rejected = IA
Rejeitar	suggested_value rejeitado, sem alternativa	Parcial: só negativo (exige nova resposta para virar par)
Tarefa descartada por regra	texto + motivo em rejected_tasks_audit	Negativo automático (sem humano)

9.2 Situação real dos dados hoje

Nas 19 reuniões analisadas há **114 sugestões** e **114 estão pendentes** (0 confirmadas, 0 editadas, 0 rejeitadas). Portanto, hoje há **zero pares de preferência**. DPO costuma pedir centenas a milhares de pares; com 44 reuniões no total, o volume atual não é suficiente nem se todas fossem revisadas. (Ordem de grandeza geral da literatura; não medi isso neste projeto.)

9.3 Duas lacunas no código para coletar bons pares (proposta)

- O update_suggestion grava *editar* com o mesmo status confirmed de *confirmar*; dá para distinguir comparando os valores, mas o schema prevê "edited" e ele nunca é usado.

- **Tarefas e dores** (o grosso da extração) não passam pelo painel Confirmar/Editar/Rejeitar; só os campos `tema`, `responsavel_reuniao`, `nivel_urgencia`, `participantes`, `justificativa_urgencia`, `status_inicial_reuniao` e as recomendações TOTVS. Sem isso não há feedback sobre o que mais importa.
- O par precisa guardar também o **contexto** (bloco da transcrição, versão do prompt, modelo). Hoje cada evento de `audit_events.json` guarda só ação, campo, valor antigo/novo e autor (sem prompt nem modelo), e o log é cortado em 1.000 eventos.

9.4 Exportador de pares (esboço, proposta)

```
import json
data = json.load(open("backend/dataset_limpo.json", encoding="utf-8"))
pairs = []
for m in data:
    for nome, sug in (m.get("field_suggestions") or {}).items():
        ia, humano = sug.get("suggested_value"), sug.get("confirmed_value")
        if sug.get("review_status") == "confirmed" and humano != ia:
            pairs.append({"prompt": f"[{nome}] " + m["ANON_TRANSCRICAO"][:6000],
                          "chosen": json.dumps(humano, ensure_ascii=False),
                          "rejected": json.dumps(ia, ensure_ascii=False)})
json.dump(pairs, open("prefs.json", "w", encoding="utf-8"), ensure_ascii=False)
```

9.5 Alternativa sem humanos: recompensa programática

O projeto já tem **verificadores automáticos** que funcionam como função de recompensa: JSON válido no schema Pydantic, evidência encontrada na transcrição (`validar_evidencias_contra_transcricao`), tarefa que passa em `classify_task_operational_intent`, prazo que o `parse_relative_deadline` reconhece. Dá para gerar N respostas por reunião, pontuá-las com essas regras e montar pares `chosen/rejected` automaticamente (técnica de *rejection sampling* / preferência por regras). Vantagem: não depende de revisão manual. Risco: o modelo aprende a agradar as regras, não necessariamente o usuário.

9.6 Como treinar e voltar para o Ollama (proposta)

- Base: `qwen2.5:3b` (cabe em máquinas sem GPU para inferência; o treino precisa de GPU ou serviço externo).
- Etapas: SFT com exemplos confirmados -> DPO com pares (Hugging Face TRL, `DPOTrainer`, LoRA) -> mesclar -> converter para GGUF -> `ollama create` com um `Modelfile` -> apontar `OLLAMA_MODEL` para ele.
- Medir antes/depois com um conjunto de reuniões separado (taxa de edição humana, evidências válidas, JSON válido).

```
from trl import DPOTrainer, DPOConfig
cfg = DPOConfig(output_dir="dpo-out", beta=0.1, learning_rate=5e-7,
                 per_device_train_batch_size=2, num_train_epochs=1)
DPOTrainer(model=model, args=cfg, train_dataset=ds, processing_class=tok).train()
```

10. Plano sugerido (proposta)

Fase	Ação	Esforço relativo	Risco
1	Determinismo: temperature 0, top_k 1, seed, schema no <i>format</i> , modelo fixo no .env	Pequeno (poucas linhas + 1 teste)	Baixo
2	Cache por hash + digest do modelo nos metadados + corrigir README	Pequeno	Baixo
3	Estender o painel de revisão a tarefas e dores; usar status "edited"; guardar prompt/modelo no evento	Médio (backend + frontend)	Baixo
4	Equipe revisa as reuniões e gera feedback real; medir % de edição	Contínuo (humano)	—
5	Exportar pares, treinar DPO com LoRA, comparar com o modelo base	Grande	Médio (qualidade e custo de GPU)

Critério de decisão: só vale ir à fase 5 se, após as fases 1-4, o modelo ainda errar com frequência em campos que os usuários corrigem. Se prompt + regras + determinismo já bastarem, DPO é custo desnecessário.

11. Checklist para o grupo

- [] Definir OLLAMA_MODEL fixo no backend/.env (hoje depende do hardware).
- [] Payload do Ollama: temperature 0, top_k 1, top_p 1, seed 42.
- [] Trocar format: "json" por JSON Schema do Pydantic.
- [] Registrar digest do modelo e versão do prompt em *analise_metadados*.
- [] Cache por hash (modelo + prompt + bloco).
- [] Teste de repetibilidade da análise.
- [] Corrigir valores dos perfis no README.
- [] Revisar sugestões pendentes (114) para começar a gerar feedback real.
- [] Decidir se tarefas/dores entram no painel Confirmar/Editar/Rejeitar.

12. Glossário

Termo	Significado
Policy	Modelo que está sendo treinado
Reward model	Modelo que dá nota à qualidade de uma resposta
PPO	Algoritmo de RL usado no RLHF
KL	Medida de distância entre o modelo novo e o de referência
SFT	Ajuste supervisionado com exemplos corretos
LoRA	Treino de poucas camadas extras; barateia o fine-tuning
Greedy decoding	Escolher sempre o token mais provável (temperature 0)
Seed	Semente do gerador aleatório
Digest (Ollama)	Hash que identifica a versão exata de um modelo

Referências: Ouyang et al., 2022 (InstructGPT/RLHF); Rafailov et al., 2023 (DPO); documentação do Ollama e do Hugging Face TRL.